SAP Build Work Zone, advanced edition

Generated on: 2026-08-08 22:10:51 GMT+0000

SAP Build Work Zone, advanced edition | Cloud

Public

Original content: https://help.sap.com/docs/WZ/b03c84105ff74f809631e494bd612e83?locale=en-US&state=PRODUCTION&version=Cloud

Warning

This document has been generated from SAP Help Portal and is an incomplete version of the official SAP product documentation. The information included in custom documentation may not reflect the arrangement of topics in SAP Help Portal, and may be missing important aspects and/or correlations to other topics. For this reason, it is not for production use.

For more information, please visit https://help.sap.com/docs/disclaimer.

UI Integration Cards

Integration cards present a new means to expose application content to the end user in a unified way.

A card is a design pattern that displays the most concise pieces of information in a limited-space container. Similar to a tile, it helps users structure their work in an intuitive and dynamic way.

Cards are composite controls that follow a predefined structure and offer content in a specific context. Cards contain the most important information for a given object (usually a task, or a list of business entities). You can use cards for presenting information, which can be displayed in flexible layouts that work well across a variety of screens and window sizes.

ℹ Note

- UI Card Editor currently supports all card features available in SAP UI5 version 1.87.0.
- All card types that are listed in SAPUI5 UI Integration Cards - Card Types are supported.

Related Info:

Step	More information
Get familiar with UI Integration cards	SAPUI5 UI Integration Cards - Overview
Get familiar with the types of cards you can develop	SAPUI5 UI Integration Cards - Card Types
Development flow	• Creating a UI Card • Updating a UI Card • Deploying or Downloading a UI Card • Developing a UI Integration Card for MTB • Creating a Design Time Module
Set up integration between cards on a page	Configuring Interaction Between Cards
Refer to the following tutorial **ℹ Note** This tutorial was created on a demo system and some steps might not match your system, however it describes the end to end flow and can be used for reference.	Create a UI5 Integration Card that Displays Data from the SAP Gateway Demo System

Creating a UI Card

Create a UI card using SAP Business Application Studio.

Prerequisites

You've created a dev space with the **Development Tools for SAP Build Work Zone** extension.

Context

This procedure explains how to create a basic card. However, you can enhance the card with many more options. For more information about the development flow, see Developing Cards.

Procedure

1. Launch SAP Business Application Studio and navigate to the dev space that you created for UI cards.
2. On the welcome screen, choose **New Project From Template**.
3. Choose **UI Integration Card** > **Next**.
4. In the **Project Details** section, provide the following details and choose **Next**.

Field name	Value
Project Name	Enter a project name.
Select a Card Sample	Select a card sample from the available samples.
NameSpace	Enter a namespace. The card ID is generated using the namespace and the project name. Card ID: <namespace>.<project name>
Card Title	Enter a title
Card Subtitle	Enter a subtitle

You've successfully created a card project that contains the required UI Card. You can now perform the following actions:

- Update the UI card: For more information, see Updating a UI Card
- Delete the UI card: To do so, navigate to the newly created card project. Right-click on the project and select **Delete**.
- Package the UI card: For more information, see Deploying or Downloading a UI Card

You can also create a UI card using the command line. To do so, navigate to **View** > **Find Command**. In the CLI, select **UI Integration Card: Create Card Project**. Provide the required details and confirm to create the project. After confirmation, the newly created project is available in your workspace.

Updating a UI Card

Update a UI card using SAP Business Application Studio.

Procedure

1. Launch SAP Business Application Studio and navigate to the dev space where you created the UI card.
2. Navigate to the card project that contains the required UI card. Right-click on the `manifest.json` file in the selected card project and choose **UI Integration Card: Edit**.

> **Note**
>
> Currently, the property editor is only supported for List, Table, and Object card types. For all other card types, use the code editor.

> **Remember**

When you edit a card, make sure you upload it to the Content Manager as a new version.

3. In the editor, update the required fields.

   To preview the changes, right-click on the `manifest.json` file and choose **UI Integration Card: Preview**. You can also select the preview button in the editor to view the changes.

   The UI Card editor provides an option to select the SAP UI5 version for card development. On selecting the SAP UI5 version, the property editor and the card preview supports features of the selected version accordingly. To select the SAP UI5 version, navigate to **File** > **Settings** > **Open Preferences** > **UIcarddk** > **ui5Version** and select the required version.

   The `autoPreviewOn ManifestChanged` and `autoPreviewOn ManifestSaved` attributes allow you to view the changes and save them automatically. To enable these attributes, set them to true by navigating to **File** > **Settings** > **Open Preferences** > **UIcarddk**.

Deploying or Downloading a UI Card

Directly deploy a UI integration card from SAP Business Application Studio to the subaccount or download it as a ZIP file.

Context

The card that you've created can be added to your subaccount as an app with a card visualization. If you have a subscription to SAP Business Application Studio in the same subaccount as your subscription to SAP Build Work Zone, advanced edition, and you have a token-exchange destination configured on it, you can directly deploy the app to the subaccount. Otherwise, you can download the card as a ZIP file, and then upload it as an app visualization in the Content Manager.

Procedure

1. **Option 1: Direct Deployment:** In SAP Business Application Studio, right-click the **manifest.json** file in the selected card project and choose **UI Integration Card: Deploy to SAP Build Work Zone**.

   Refresh the card editor in the in the Admin Console, **UI Integration** > **Cards**. The card is added to the list of cards and you can activate it.

   In the Content Manager, you can find a new local app with this card visualization.

   ### ℹ Note

   - On the first deployment of the sample card, it's recommended to clear the destination. To do so, navigate to **File** > **Settings** > **Open Preference** > **UIcarddk**. In the **UIcarddk** section, remove the value available for **Content Destination**.
   - For successive deployments of the sample card, it isn't required to clear the content destination.

2. **Option 2: Downloading a Zip File:** In SAP Business Application Studio, right-click the **manifest.json** file in the selected card project and choose **UI Integration Card: Package**.

   The UI card is packaged in a `<card id>.zip` file. Right-click the `<card id>.zip` file and select **Download** to download the file.

   In the Content Manager, create a new local app and upload the card ZIP file as the app visualization. For more information, see Configuring Apps with a Card Visualization, Integrating Cards into a Site.

Developing a UI Integration Card for MTB

Develop a UI card for MTB (Mobile Transaction Bridge) backend using SAP Business Application Studio.

Context

The following procedure describes steps on how to create, update, and deploy a UI card from MTB generated OData services.

Procedure

1. Launch SAP Business Application Studio and navigate to the dev space that you created for UI cards.
2. On the Get Started page, choose **New Project From Template**.
3. Choose **UI Integration Card For MTB** > **Start**.
4. In the **Project Details** section, provide the following details and choose **Next**.

Field name	Value
Project Name	Enter a project name.
NameSpace	Enter a namespace. The card ID is generated using the namespace and the project name. Card ID: <namespace>.<project name>
Select a Card Type	Select a card type from the available samples. . **Note** Currently, the supported card types are: **Object Card**, **List Card**, and **Table Card**.
Title	Enter a title
Subtitle	Enter a subtitle

5. In **MTB Backend** section, provide the following details and choose **Next**.

Field name	Value
Select a Destination	Select a destination from the dropdown.
Select an Application	Select an application from the dropdown.
Select a Data Source	Select a data source from the dropdown.
Select a Function	Select a function from the dropdown.

6. In **Data Setting** section, provide the following details and choose **Next**. In this step, you must provide the value for each available parameter for the MTB function specified in the previous step.

Field name	Value
Value for Parameter <parameter name>	Enter a value for the parameter.
Do you want to generate card configuration and content automatically?	Select either **Yes** or **No**. If you chose yes, then the card content is generated automatically and the required UI card is created.

Field name	Value
	If you chose no, then proceed to the next step to create the card content manually.

7. Based on the selected card type, in the **Card Configuration** section, specify the parameters, header type, and other configurations related to the card type. The provided values are included in the card configuration.
8. In **Card Content** section, specify the detailed data binding as required.

Creating a Design Time Module

A new design time module is used to configure and implement functionality used in the card editor. The editor is launched by the host environment for different personas and uses the design time module to create the user interface.

When the user creates a new UI Card project using the **Yeoman** generator, the design time module is already available in the generated project. ("dt" folder and the inner "configuration.js" file are ready, and "designtime" property is specified as "dt/configuration" in the manifest.json)

For existing projects without a design time module setup, when the user opens the property editor, the design time module is set up automatically.

Create a design time module and register it in the cards `manifest.json` file

> **Note**
>
> Advanced design time configuration and implementation should be defined outside the `manifest.json` file of the card so that it won't harm or influence the runtime of the card instance for the end user.

Add the following initial module setup into the `dt/configuration.js` file.

```
sap.ui.define(["sap/ui/integration/Designtime"], function (
        Designtime
) {
        "use strict";
        return function () {
                return new Designtime({
                        form: {
                                items: {
                                        //here goes the configuration
                                }
                        },
                        preview: {
                                modes: "Abstract"
                        }
                });
        };
});
```

Similar to a card extension, the design time module is registered in the `manifest.json` file as soon as the editor is launched.

```
"sap.card": {
        "designtime": "dt/configuration"
```

```
}
```

Configure the editor's form

Within the configuration, the form section contains the items that are shown in the editor.

```
sap.ui.define(["sap/ui/integration/Designtime"], function (
        Designtime
) {
        "use strict";
        return function () {
                return new Designtime({
                        form: {
                                items: {

                                }
                        },
                        preview: {
                                modes: "Abstract"
                        }
                });
        };
});
```

Form items

Field Type Items

Property	Type	Required	Default	Description
manifestpath	string	Yes		Path to the manifest that should be edited. In case the type of the item is `grou` Example: `manifestpath : "sap.card/configuration/parameters`
type	string	Yes	string	The type of the value in the manifest that is edited. Currently "string", "integer" "boolean", "string[]" are supported. `type="group"` in an item allows to set a group title within the form to cluster manifestpath can be omitted and a label can be provided instead. Example: `type : "string"`
label	string	No	Name of the item in the form/items collection	Defines the label string. This value can be bound to values in the i18n file used i Example: `label : "fixedString` or `label : "{i18n>translatedSt`

defaultValue	any	No	Depending on type property	Defines the default value if no value is specified in the manifest.
				
				Example:
				
				`defaultValue : "stringValue" //string type`
				`defaultValue : true //boolean type`
				`defaultValue : "{i18n>translatedDefaultKey}" //trnaslated c`

required	boolean	No	false	Defines whether the value is required or not. The editors label for the field gets

Property	Type	Required	Default	Description
				Example: `required: true`
visible	boolean		true	Defines whether the value is visible in the editor or not. For technical paramete them from the user. Example: `visible: true`
translatable	boolean	No	false	Defines whether the value is translatable and should be shown in the card edito setting should be used for fields of type string only. Example: `translatable: true`
cols	1 or 2	No	2	By default, the form in the editor spans fields in two columns. Setting `cols=1` align two fields next to each other. Example: `cols: 1`
allowDynamicValues	boolean	No	depending	Defines whether the value can be bound to a context value of the host environn administrator and content admin is true. In translation mode this property is ig the `allowDynamicValues` property of the card editor control needs to be er Example: `allowDynamicValues: true`
allowSettings	boolean	No	depending	Defines whether the administrator is allowed to change the settings of the field can hide or disable fields for the content mode. To activate the feature, the `all` card editor control needs to be enabled. Example: `allowSettings: true`

Providing a list of values

To allow a selection of values for fields of type "string", you can add a value section. Similar to the card data section, the list can be filled with a static json or a request. The `item` section in the value definition links the data. The `key` property of the item is used for the setting referred by the `manifestPath`.

Sample Code

Static Data List:

```
"stringWithStaticList": {
  "manifestpath": "/sap.card/configuration/parameters/stringWithStaticList/value",
  "type": "string",
  "values": {
     "data": {
        "json": {
           "values": [
              { "text": "From JSON 1", "key": "key1" },
              { "text": "From JSON 2", "key": "key2" },
              { "text": "From JSON 3", "key": "key3" }
           ]
        },
        "path": "/values"
     },
     "item": {
        "text": "{text}",
        "key": "{key}"
     }
```

```
    }
  }
```

Request Data List: The referred URL delivers the data asynchronously. For the data provider you can also use an extension.

Sample Code

Request Values

```
"stringWithStaticList": {
  "manifestpath": "/sap.card/configuration/parameters/stringWithRequestList/value",
  "type": "string",
  "values": {
     "data": {
        "request": {
            "url": "./dt/listdata.json"
        },
        "path": "/values"
     },
     "item": {
        "text": "{text}",
        "key": "{key}"
     }
  }
}
```

Sample Code

Data of dt/listdata.json

```
"values": [
                { "text": "From JSON 1", "key": "key1" },
                { "text": "From JSON 2", "key": "key2" },
                { "text": "From JSON 3", "key": "key3" }
        ]
```

Providing Lists for string arrays

To allow a selection of values for fields of type "string[]", you can add the same values sections as above. Similar to the card data section, the list can be filled with a static json or a request. The `item` section in the values definition links the data. The `key` property of the item is used as the value for the setting referred by the `manifestPath`.

Sample Code

Static Array List:

```
"stringArrayWithStaticList": {
        "manifestpath": "/sap.card/configuration/parameters/stringArrayWithStaticList/value",
        "type": "string[]",
        "values": {
                "data": {
                "json": {
```

```
                        "values": [
                                { "text": "From JSON 1", "key": "key1" },
                                { "text": "From JSON 2", "key": "key2" },
                                { "text": "From JSON 3", "key": "key3" }
                        ]
                },
                "path": "/values"
                },
                "item": {
                "text": "{text}",
                "key": "{key}"
                }
        }
}
```

Request Data List: The referred URL delivers the data asynchronously. For the data provider you can also use an extension.

Sample Code

String Array List with requests

```
"stringWithStaticList": {
        "manifestpath": "/sap.card/configuration/parameters/stringArrayWithRequestList/value",
        "type": "string[]",
        "values": {
                "data": {
                "request": {
                        "url": "./dt/listdata.json"
                },
                "path": "/values"
                },
                "item": {
                "text": "{text}",
                "key": "{key}"
                }
        }
}
```

Sample Code

Data of dt/listdata.json

```
        "values": [
                { "text": "From JSON 1", "key": "key1" },
                { "text": "From JSON 2", "key": "key2" },
                { "text": "From JSON 3", "key": "key3" }
        ]
```

Configuring Interaction Between Cards

Cards on the same workpage can interact with each other via the page context.

When developing cards, you can add parameters to the cards that write and read to the page context. At runtime, if the workpage contains several cards that are configured to respond to the page context, a user can select a context value in one card and this will update the other cards according to the selected value.

To learn more about how to configure card integration via a page context, refer to the following Developer Guide in Github. The guide contains an example of three cards that interact with each other:

- **Brand card** - Lists all available brands in a dropdown list. Once a brand is selected, it is added to the current card context as the brand parameter.
- **Region card** - Lists all available regions in a dropdown list. Once a region is selected, it is added to the current card context as the region parameter.
- **Analytical card** - A card that subscribes to the brand and region card context parameters. The analytical chart automatically refreshes when the brand or region parameter is updated.

For more information, see:

- Interaction Between Cards in the SAP Build Work Zone
- Build-In Card Context
- Sample cards :

Supported Declarative and Component Card Features

An overview of card features and whether they are supported on different SAP Build Work Zone editions.

Background

UI integration cards consist of different card types, such as declarative cards, component cards, etc. For more information see, Card Types.

While declarative cards are fully supported in both SAP Build Work Zone, advanced edition and SAP Build Work Zone, standard edition, this is not the case in component cards. Some component card features are only supported in SAP Build Work Zone, advanced edition.

This gap becomes an issue when cards are uploaded as app visualizations in SAP Build Work Zone, standard edition. If the uploaded card contains a reference to a feature that is only supported in SAP Build Work Zone, advanced edition, the card will fail to render at runtime. This issue affects both SAP Build Work Zone, standard edition Web client and its mobile client on the Joule Work mobile app.

Note

In design-time, the card visualization will be available in the content gallery and will be displayed inside the page editor. However, the card will fail to render at runtime and will result in an error.

Supported & Not Supported Card Features

The following table provides information about different card features and whether they are supported on different clients.

Card Feature	SAP Build Work Zone, advanced edition (Web)	SAP Build Work Zone, standard edition (Web)	Joule Work Mobile App
Declarative Card	Yes	Yes	Yes
Declarative Card with Extension	Yes	Yes	Yes
Component Card	Yes	Yes	No - filtered out
Read context variables for **all editions**: • `sap.workzone.currentUser`	Yes	Yes	No
Read context variables for SAP Build Work Zone, advanced edition: • `global_uuid`, `user_type`, `time_zone`, `sap.workzone.currentUser`, `sap.workzone.currentWorkspace`, `sap.workzone.currentCompany`	Yes	No	No
Read context variables for SAP SuccessFactors Work Zone and SAP Build Work Zone, advanced edition contexts: • `sap.successfactors.*`	Yes	No	No
Read/Add/Change/Refresh context variables	Yes	No	No

ℹ Note

- Read context support means that the host environment supports the following methods: `getContextValue` and `getContext`.
- Conext Awareness (last item) means that the host environment also supports the method `updateContext`.